DATABASE MANAGEMENT SYSTEMS | 23A05402T

UNIT – II : Relational Model & Basic SQL

Easy handwritten notes with diagrams for students

Topics in this unit

1. Relational model: domain, attribute, tuple, relation
2. Importance of null values
3. Constraints: domain, key and integrity constraints
4. Relational algebra (operators with examples)
5. Relational calculus (tuple and domain)
6. Basic SQL: schema, data types, CREATE / ALTER
7. DML operations: INSERT, DELETE, UPDATE

* **Study tip:** Every topic has a small diagram or table. Learn the **RED words** first, then draw the diagram once from memory – that is the fastest way to remember it for exams.

Syllabus (as given)

Relational Model: Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra, Relational Calculus. BASIC SQL: Simple Database schema, data types, table definitions (create, alter), different DML operations (insert, delete, update).

Part I : The Relational Model

1.1 Introduction

The **relational model** was proposed by **E.F. Codd (1970)**. It stores all data in simple two-dimensional **tables** called relations. It is the basis of almost every modern DBMS (MySQL, Oracle, PostgreSQL, SQL Server). Its strength is simplicity, a strong mathematical base (set theory, logic) and the standard language SQL.

1.2 Basic Terms

Term	Meaning	Table word
Relation	A table with a name; a set of tuples	Table
Tuple	One row = one record of the relation	Row / record
Attribute	A named column; property of the relation	Column / field
Domain	Set of allowed (atomic) values for an attribute	Data type + range
Degree (arity)	Number of attributes (columns)	No. of columns
Cardinality	Number of tuples (rows)	No. of rows
Relation schema	Name + list of attributes: STUDENT(Roll_no, Name, Age, Dept_id)	Table design
Relation instance	The set of tuples present at a given time	Table data

STUDENT (relation)

Attribute (column)

Roll_no	Name	Age	Dept_id
101	Ravi	20	D1
102	Sita	19	D2
103	Arun	21	D1

Tuple (row)

Degree = 4 , Cardinality = 3 , Domain of Age = integers (e.g. 15 to 60)

Fig 2.1 : A relation with its parts

Properties of a relation

- Relation name is unique; attribute names in a relation are unique.
- Every value is **atomic** (single, not a list) - this is first normal form.
- **No duplicate tuples** (a relation is a set).
- **Order of rows and columns does not matter**.

1.3 Importance of NULL Values

NULL is a special marker meaning the value is **unknown**, **missing** or **not applicable**. It is NOT zero and NOT an empty string.

Use of NULL	Example
Value unknown	A student's phone number is not yet collected
Not applicable	Spouse name for an unmarried employee
Value exists but missing	Date of birth not recorded

- Any arithmetic with NULL gives NULL (**5 + NULL = NULL**).

- Comparison with NULL gives **UNKNOWN** (third truth value) - so use **IS NULL / IS NOT NULL**, never **= NULL**.
- Aggregates (SUM, AVG, COUNT(col)) ignore NULLs; **COUNT(*)** counts all rows.
- Primary key attributes can **never** be NULL.
- Importance: lets the DB store incomplete information without inventing fake values.

A	B	A AND B	A OR B	NOT A
TRUE	UNKNOWN	UNKNOWN	TRUE	FALSE
FALSE	UNKNOWN	FALSE	UNKNOWN	TRUE
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

Part 2 : Keys and Constraints

2.1 Keys

Key	Meaning	Example (STUDENT)
Super key	Any set of attributes that uniquely identifies a tuple	{Roll_no}, {Roll_no, Name}, {Email}
Candidate key	Minimal super key (no extra attribute)	Roll_no, Email
Primary key	Candidate key chosen by designer; not NULL; unique	Roll_no
Alternate key	Candidate keys not chosen as primary	Email
Composite key	Key made of two or more attributes	(Roll_no, Course_id) in ENROLLS
Foreign key	Attribute that refers to the primary key of another (or same) table	Dept_id in STUDENT

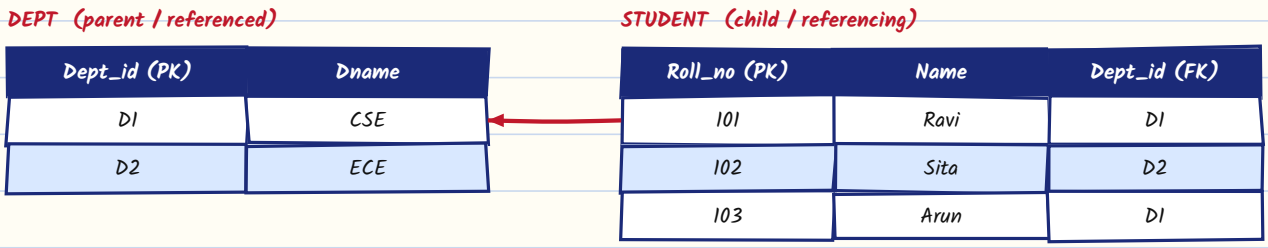


Fig 2.2 : Foreign key in STUDENT refers to primary key of DEPT

2.2 Integrity Constraints and their Importance

Constraints are rules that every instance of the database must obey. The DBMS checks them automatically on INSERT, UPDATE and DELETE, so wrong data never enters the database.

Constraint	Rule	Why important
Domain constraint	Value must come from the attribute's domain (data type, range, NOT NULL, CHECK)	Age cannot be -5 or 'abc'
Key constraint	Key attributes must be unique (UNIQUE / PRIMARY KEY)	No two students share a Roll_no
Entity integrity	Primary key cannot be NULL	Every row can be identified
Referential integrity	A foreign key value must either be NULL or match an existing primary key value in the parent table	No student belongs to a non-existing department

Actions to keep referential integrity (ON DELETE / ON UPDATE)

Option	Meaning
RESTRICT / NO ACTION	Reject the delete/update if child rows exist
CASCADE	Delete / update the child rows also
SET NULL	Set the foreign key in child rows to NULL
SET DEFAULT	Set the foreign key to its default value

* **Remember:** Entity integrity -> **primary key not null**. Referential integrity -> **foreign key matches primary key**. Domain -> **valid value**.

Part 3 : Relational Algebra

Relational algebra is a **procedural query language**: you tell the system step by step **how** to get the result. Each operator takes one or two relations and returns a new relation. It is the theoretical base of SQL.

Sample relations used below: STUDENT(Roll_no, Name, Age, Dept_id) with rows (101 Ravi 20 D1), (102 Sita 19 D2), (103 Arun 21 D1); DEPT(Dept_id, Dname) with rows (D1 CSE), (D2 ECE).

3.1 Fundamental (Basic) Operators

Operator	Symbol / syntax	Meaning and example
Select	$\sigma_{condition}(R)$	Chooses ROWS that satisfy the condition. $\sigma_{Age>19}(STUDENT)$ -> Ravi, Arun.
Project	$\pi_{A1,A2}(R)$	Chooses COLUMNS, removes duplicates. $\pi_{Name,Age}(STUDENT)$.
Union	$R \cup S$	All tuples in R or S or both (no duplicates). Needs union-compatible relations.
Set difference	$R - S$	Tuples in R but not in S.
Cartesian product	$R \times S$	Every tuple of R paired with every tuple of S; degree = sum, rows = m x n.
Rename	$\rho_X(R)$	Gives a new name to relation or attributes.

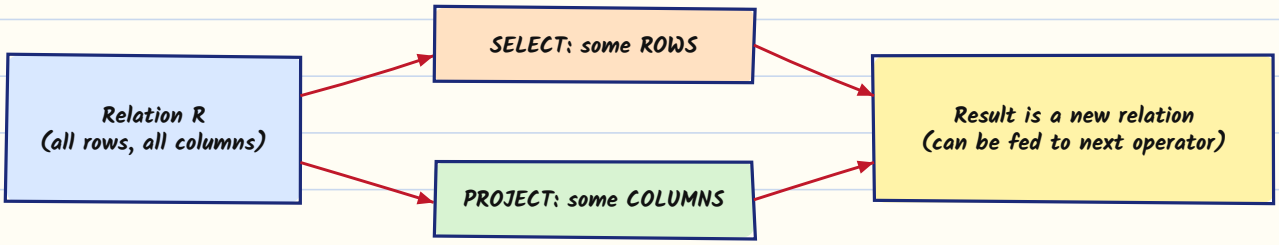


Fig 2.3 : Select picks rows, Project picks columns

* **Union-compatible:** Two relations must have the **same number of attributes** with **matching domains** for union, intersection and difference.

3.2 Additional Operators

Operator	Symbol	Meaning
Intersection	$R \cap S$	Tuples common to R and S. $R \cap S = R - (R - S)$.
Theta join	$R \bowtie_{\theta} S$	Cartesian product followed by select with condition θ (any comparison).
Equi-join	$R \bowtie_{R.A=S.B} S$	Theta join where condition uses only equality.
Natural join	$R \bowtie S$	Equi-join on all common attributes; the common column appears once.
Left / Right / Full outer join	$R \bowtie_{L} S$, $R \bowtie_{R} S$, $R \bowtie_{F} S$ (L = left, R = right, F = full)	Join that also keeps unmatched tuples of left / right / both, filling NULLs.
Division	$R \div S$	Tuples of R related to ALL tuples of S ("for all" queries).
Aggregate	$\mathcal{F}$ / G	SUM, AVG, MIN, MAX, COUNT, e.g. $_{Dept_id}\mathcal{G}_{COUNT(Roll_no)}(STUDENT)$.

Example: Natural join STUDENT ⋈ DEPT

Roll_no	Name	Age	Dept_id	Dname
101	Ravi	20	D1	CSE
102	Sita	19	D2	ECE
103	Arun	21	D1	CSE

Example queries

Query in English	Relational algebra
Names of students older than 19	$\pi_{Name}(\sigma_{Age>19}(STUDENT))$
Names of students of CSE	$\pi_{Name}(\sigma_{Dname='CSE'}(STUDENT \bowtie DEPT))$
Students in D1 and Age > 20	$\sigma_{Dept_id='D1' \wedge Age>20}(STUDENT)$
Departments with no students	$\pi_{Dept_id}(DEPT) - \pi_{Dept_id}(STUDENT)$

Exam tip: Select σ = **rows** (WHERE), Project π = **columns** (SELECT list). Only the six basic operators (σ, π, ∪, −, ×, ρ) are truly needed; others can be derived from them.

Part 4 : Relational Calculus

Relational calculus is a **non-procedural (declarative)** language based on predicate logic: you describe **what** result you want, not how to get it. Its expressive power equals relational algebra for safe expressions (Codd's theorem).

Point	Tuple Relational Calculus (TRC)	Domain Relational Calculus (DRC)
Variable ranges over	Whole tuples (rows)	Individual attribute values (domains)
General form	$\{ t \mid P(t) \}$	$\{ \langle x1, x2, \ldots, xn \rangle \mid P(x1, \ldots, xn) \}$
Example: students with Age > 20	$\{ t \mid t \in STUDENT \wedge t.Age > 20 \}$	$\{ \langle r, n, a, d \rangle \mid \langle r, n, a, d \rangle \in STUDENT \wedge a > 20 \}$
Example: names only	$\{ t.Name \mid t \in STUDENT \wedge t.Age > 20 \}$	$\{ \langle n \rangle \mid \exists r,a,d\ (\langle r,n,a,d \rangle \in STUDENT \wedge a > 20) \}$
Basis of	SQL	QBE (Query-By-Example)

Symbols used

Symbol	Meaning	Symbol	Meaning
∧	AND	∃	there exists (at least one)
∨	OR	∀	for all
¬	NOT	⇒	implies

- **Free variable** - appears in the result; **bound variable** - quantified by ∃ or ∀.
- **Safe expression** - guarantees a finite result (only values from the database). Unsafe e.g. { t | ¬(t ∈ STUDENT) }.

Relational algebra	Relational calculus
Procedural - how to get result	Declarative - what result is needed
Uses operators on relations	Uses logic predicates and quantifiers
Order of operations matters	No order of operations
Base for query optimisation	Base for SQL (TRC) and QBE (DRC)

Part 5 : Basic SQL

5.1 What is SQL?

SQL (Structured Query Language) is the standard language of relational databases (ANSI / ISO). It is declarative and not case-sensitive for keywords. Statements end with a semicolon.

Category	Commands	Purpose
DDL	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define / change structure (schema)
DML	INSERT, UPDATE, DELETE (SELECT)	Change / read the data
DCL	GRANT, REVOKE	Access permissions
TCL	COMMIT, ROLLBACK, SAVEPOINT	Control transactions

5.2 Simple Database Schema (used in examples)

```
DEPT    (Dept_id PK, Dname, Location)
STUDENT (Roll_no PK, Name, Age, Email, Dept_id FK -> DEPT, Fees)
COURSE  (Course_id PK, Title, Credits)
ENROLLS (Roll_no FK, Course_id FK, Grade)   PK = (Roll_no, Course_id)
```

5.3 Data Types

Type	Description	Example
INT / INTEGER	Whole numbers	101
SMALLINT / BIGINT	Small / large whole numbers	32000
DECIMAL(p,s) / NUMERIC	Exact number, p digits, s after point	DECIMAL(8,2) = 12345.67
FLOAT / REAL / DOUBLE	Approximate decimal number	3.14
CHAR(n)	Fixed length string (padded)	CHAR(2) = 'D1'
VARCHAR(n)	Variable length string up to n	'Ravi Kumar'

Type	Description	Example
TEXT	Very long text	address, notes
DATE / TIME / TIMESTAMP	Date, time, date+time	'2026-10-15'
BOOLEAN	TRUE / FALSE	TRUE
BLOB	Binary large object (image, file)	photo

5.4 Table Definition : CREATE

```
CREATE TABLE DEPT (
    Dept_id   CHAR(2)      PRIMARY KEY,
    Dname     VARCHAR(30)  NOT NULL UNIQUE,
    Location  VARCHAR(30)  DEFAULT 'Main Campus'
);

CREATE TABLE STUDENT (
    Roll_no  INT          PRIMARY KEY,
    Name     VARCHAR(40)   NOT NULL,
    Age      INT          CHECK (Age BETWEEN IS AND 60),
    Email    VARCHAR(50)   UNIQUE,
    Dept_id  CHAR(2),
    Fees     DECIMAL(8,2)  DEFAULT 0,
    FOREIGN KEY (Dept_id) REFERENCES DEPT(Dept_id)
       ON DELETE SET NULL ON UPDATE CASCADE
);
```

Column constraint	Effect
PRIMARY KEY	Unique + NOT NULL; identifies the row
NOT NULL	Value must be given
UNIQUE	No duplicate values (NULL allowed)
CHECK (cond)	Value must satisfy the condition (domain constraint)
DEFAULT v	Value used when none is supplied
FOREIGN KEY ... REFERENCES	Referential integrity link to another table

5.5 Table Definition : ALTER, DROP, TRUNCATE

```
ALTER TABLE STUDENT ADD Phone VARCHAR(15);          -- add column
ALTER TABLE STUDENT MODIFY Name VARCHAR(60);         -- change type (MySQL/Oracle)
ALTER TABLE STUDENT ALTER COLUMN Name TYPE VARCHAR(60);  -- PostgreSQL style
ALTER TABLE STUDENT DROP COLUMN Phone;               -- remove column
ALTER TABLE STUDENT ADD CONSTRAINT chk_age CHECK (Age >= 15);
ALTER TABLE STUDENT DROP CONSTRAINT chk_age;
ALTER TABLE STUDENT RENAME TO STUDENTS;              -- rename table

TRUNCATE TABLE STUDENT;   -- delete ALL rows, keep structure (fast, no rollback in most DBs)
DROP TABLE STUDENT;       -- delete structure + data permanently
```

Command	Removes	Structure kept?	WHERE allowed?
DELETE	Selected / all rows	Yes	Yes
TRUNCATE	All rows	Yes	No

Command	Removes	Structure kept?	WHERE allowed?
DROP	Whole table	No	No

5.6 DML Operations : INSERT, UPDATE, DELETE

INSERT – add new rows

```
INSERT INTO DEPT VALUES ('D1', 'CSE', 'Block A');
INSERT INTO STUDENT (Roll_no, Name, Age, Dept_id)      -- only some columns
      VALUES (101, 'Ravi', 20, 'D1');
INSERT INTO STUDENT VALUES (102,'Sita',19,'s@x.com','D2',5000);
INSERT INTO STUDENT (Roll_no, Name, Age)               -- many rows at once
      VALUES (103,'Arun',21), (104,'Meena',22);
INSERT INTO ALUMNI SELECT * FROM STUDENT WHERE Age > 22;   -- from a query
```

UPDATE – change existing rows

```
UPDATE STUDENT SET Age = 21 WHERE Roll_no = 101;
UPDATE STUDENT SET Fees = Fees * 1.10 WHERE Dept_id = 'D1';
UPDATE STUDENT SET Dept_id = 'D2', Age = Age + 1 WHERE Roll_no = 103;
```

DELETE – remove rows

```
DELETE FROM STUDENT WHERE Roll_no = 104;
DELETE FROM STUDENT WHERE Age < 18;
DELETE FROM STUDENT;          -- removes ALL rows (table remains)
```

* **Careful:** UPDATE or DELETE without a **WHERE** clause affects **every row**. Inserting a duplicate primary key, or a foreign key that does not exist, is rejected by the DBMS.

Quick Revision - Unit II

- Relational model (Codd, 1970): data in tables. Relation = table, tuple = row, attribute = column, domain = allowed values.
- Degree = number of columns; cardinality = number of rows.
- NULL = unknown / not applicable; comparisons give UNKNOWN; use IS NULL.
- Keys: super, candidate, primary, alternate, composite, foreign.
- Constraints: domain, key, entity integrity (PK not null), referential integrity (FK matches PK).
- Algebra (procedural): σ select rows, π project columns, ∪, −, ×, ρ basic; ∩, ⋈, ÷ derived.
- Calculus (declarative): TRC {t | P(t)} and DRC { | P}; ∃ and ∀.
- SQL: DDL (CREATE, ALTER, DROP), DML (INSERT, UPDATE, DELETE), DCL, TCL.
- CREATE TABLE with PRIMARY KEY, NOT NULL, UNIQUE, CHECK, DEFAULT, FOREIGN KEY.
- DELETE removes rows, TRUNCATE removes all rows, DROP removes the table.

Possible exam questions

1. Explain the relational model with terms: domain, attribute, tuple, relation, degree, cardinality.
2. What are NULL values? Why are they important? How do they behave in comparisons?
3. Explain the different keys with examples.
4. Explain domain, key, entity integrity and referential integrity constraints and their importance.
5. Explain the operators of relational algebra with examples.
6. Differentiate tuple and domain relational calculus; algebra vs calculus.
7. Write the SQL statements to create tables with constraints, and alter them.
8. Explain INSERT, UPDATE and DELETE with examples. Differentiate DELETE, TRUNCATE, DROP.